Procura: Scheduling Personal Tasks with Operating-System Policies, Grounded AI Diagnosis, and Self-Healing Recovery

System documentation and evaluation

[Your Name]
[Department, Institution]
Source code: github.com/harshitchhabi/procura

Abstract

Personal task managers order work by static, user-assigned labels, never measure whether that ordering is fair, and cannot explain or repair a stalled task. Procura treats each task as an operating-system process and applies production scheduling, resource-management and self-healing techniques to a to-do list. It implements six CPU-scheduling policies, including EEVDF (the default Linux scheduler since kernel 6.6) and the Multi-Level Feedback Queue; a five-term effective-priority score with Slurm-style decaying fair-share; Jain's Fairness Index for every schedule; a simulated resource manager with lock-order checking, timeout-based abort and wait-for-graph deadlock detection; and a detect-diagnose-recover loop that uses NIST statistical process-control detectors, a retrieval-augmented language model grounded in recorded incident facts, and deterministic recovery scripts that follow Kubernetes-style preemption rules. On a seeded 30-job Poisson workload all six policies are work-conserving (makespan 228.67 minutes, full utilisation); SJF gives the lowest average waiting time (34.0 minutes against 61.6 for FCFS) at the cost of a higher maximum wait, while Round Robin and MLFQ cut average response time on a live seven-task demonstration from 431.4 minutes (FCFS) to 11.9. Verification against the running system uncovered twelve defects, including one that had inverted the published SJF result and two that caused the assistant to invent causes and definitions; all are fixed and covered by an 83-test suite. The paper documents every subsystem as implemented and reports these findings and the system's remaining limitations.

Keywords: *CPU scheduling; EEVDF; multi-level feedback queue; fairness; deadlock handling; AIOps; retrieval-augmented generation; self-healing systems.*

Contents

1. Introduction 3
2. Background and related work 3
3. System overview 5
4. Process model 6
5. Scheduling 7
6. Resource management and deadlock handling 8
7. Detection, diagnosis and recovery 9
8. Interfaces and configuration 11
9. Verification 12
10. Evaluation 14
11. Limitations and threats to validity 16
12. Future work 17
13. Conclusion 17
References 18
Appendix A. Reproduction 19

1. Introduction

An operating system must repeatedly decide which of several runnable processes gets a scarce shared resource next, how to keep that decision fair over time, and what to do when processes contend for resources or fail. Decades of research have produced precise answers: scheduling policies with known fairness and starvation properties, metrics for comparing them, mechanisms for avoiding or breaking deadlock, and, more recently, automated detect-diagnose-remediate loops in site-reliability engineering.

Personal task management rarely borrows any of this. A typical tool asks the user to label tasks High, Medium or Low and sorts by the label. Nothing combines urgency, waiting time and cost into an order; nothing reports whether attention is distributed fairly; and when a task stalls because a shared resource (time, budget, a reviewer) is exhausted, the tool shows a flag rather than an explanation or a remedy.

Procura is a local-first web application that closes this gap by modelling each task as a process and running real scheduling, resource-management and recovery mechanisms over it. Its design was informed by a structured survey of production schedulers, AIOps practice and the statistics and concurrency-control literature, so that each subsystem corresponds to a documented real-world mechanism rather than a metaphor.

This paper is the system's technical documentation. Its contributions are:

- **C1.** A process model for tasks with an enforced six-state lifecycle, persisted in SQLite and exposed through a 16-endpoint REST API (Sections 3–4).
- **C2.** Six scheduling policies over the same task set — FCFS, SJF, Priority, Round Robin, EEVDF and MLFQ — with a uniform metric set including Jain's Fairness Index, and a five-term effective-priority score with Slurm-style fair-share decay (Section 5).
- **C3.** A simulated resource manager covering the practical spectrum of deadlock handling: lock-order checking, timeout-based abort, wait-for-graph cycle detection, and Banker's Algorithm as a textbook baseline (Section 6).
- **C4.** A detect-diagnose-recover engine with three NIST statistical detectors, retrieval-augmented diagnosis grounded in recorded incident facts, and deterministic recovery with Kubernetes-style preemption rules (Section 7).
- **C5.** A reproducible benchmark over synthetic Poisson workloads and Standard Workload Format traces, with results for all six policies (Section 10).
- **C6.** A verification account: twelve defects found by exercising the running system, their root causes, and the tests that now guard them (Section 9).

2. Background and related work

2.1 Production CPU scheduling

Linux used the Completely Fair Scheduler (CFS) from kernel 2.6.23 (2007) until EEVDF replaced it in kernel 6.6 [1, 2]. CFS keeps runnable tasks in a red-black tree ordered by virtual runtime and always runs the task with the smallest; virtual runtime grows in inverse proportion to a weight derived from the nice value, approximately `1024 / 1.25^nice` [3]. EEVDF (Earliest Eligible Virtual Deadline First) tracks each task's lag against an ideal fair-share schedule, treats tasks with non-negative lag as eligible, and dispatches the eligible task with the earliest virtual deadline [1, 2]. The Multi-Level Feedback Queue of *Operating Systems: Three Easy*

Pieces infers a job's behaviour through five rules — strict priority between levels, round robin within a level, new jobs at the top, demotion after a time allotment, and a periodic priority boost that bounds starvation [4]. FCFS, SJF and static priority have no starvation bound, and non-preemptive policies suffer the convoy effect [5].

2.2 Fairness and job schedulers

Jain, Chiu and Hawe's fairness index, $J = (\Sigma x_i)^2 / (n \cdot \Sigma x_i^2)$, ranges from 1/n to 1 [6, 7]. Slurm computes job priority as a weighted sum of normalised factors whose fair-share term penalises accounts that have consumed more than their share, with usage decaying over a configurable half-life [8], and backfills lower-priority jobs only when doing so delays no higher-priority job [9]. The Kubernetes scheduler is a pluggable framework that treats priority (which drives preemption) separately from placement, and its default preemption evicts only lower-priority pods [10, 11]. Celery's ecosystem favours a few discrete priority tiers over fine-grained numeric priority [12]. Bug-triage research and Google's Borg are precedents for applying OS-style scheduling to non-OS work [40]; WSJF in SAFe is a weighted shortest-job-first ranking [44].

2.3 AIOps, self-healing and LLM root-cause analysis

Google's SRE practice defines toil as manual, repetitive, automatable work and treats automating it as a core goal [13]. Kubernetes controllers formalise the underlying pattern as a non-terminating control loop that moves current state towards desired state [14]. Commercial platforms pair statistical anomaly detection with causal diagnosis: Datadog Watchdog [15], Dynatrace Davis AI [16] and New Relic's applied intelligence [17]. Recent work applies language models to root-cause analysis: RCACopilot [18], in-context learning with GPT-4 over more than 100,000 incidents [19], a ReAct-style agent [20], and SynergyRCA, which combines retrieval over graph-structured state with GPT-4o and criticises flat, topology-blind document retrieval [21]. Remediation in practice runs through named, auditable runbooks [22, 23], with AWS classifying each step's impact so that mutating steps can require review [24].

2.4 Statistical anomaly detection

The NIST/SEMATECH handbook specifies the exponentially weighted moving average (EWMA) control chart [25], the median-and-MAD-based modified Z-score, which it recommends over the naive Z-score rule [26], and the CUSUM change detector [27]. The same family underlies monitoring tools such as Prometheus [29]; applying it to IT metrics needs a rolling baseline because their mean is rarely stable [28].

2.5 Deadlock handling in practice

Banker's Algorithm requires advance knowledge of every process's maximum claim and a fixed population, and has essentially no production use [30]. Real systems span a cost-precision spectrum: wound-wait timestamp ordering in Spanner [31], fixed lock ordering checked by Linux's lockdep [32], timeout-based abort in MySQL InnoDB [33], and timer-gated wait-for-graph detection in PostgreSQL [34]. Chandy, Misra and Haas give the classic distributed detection algorithm [35].

2.6 Evaluating schedulers

The Parallel Workloads Archive defines the 18-field Standard Workload Format (SWF) and hosts about forty real logs [36]; the Lublin–Feitelson model is the standard synthetic generator [37]. Makespan, turnaround, waiting time and utilisation are the usual metrics [38, 39]; Google's cluster traces [41] and simulators such as

CloudSim [42] are common in larger studies. Starvation has no canonical definition and is usually operationalised as a waiting-time threshold [43].

3. System overview

Figure 1 shows the architecture. A React frontend and a terminal dashboard (`procura_top`) both talk to a single FastAPI backend, which coordinates four subsystems and persists all state in one SQLite database. The RAG engine is called only by the error-and-recovery engine and the `/ask` endpoint.

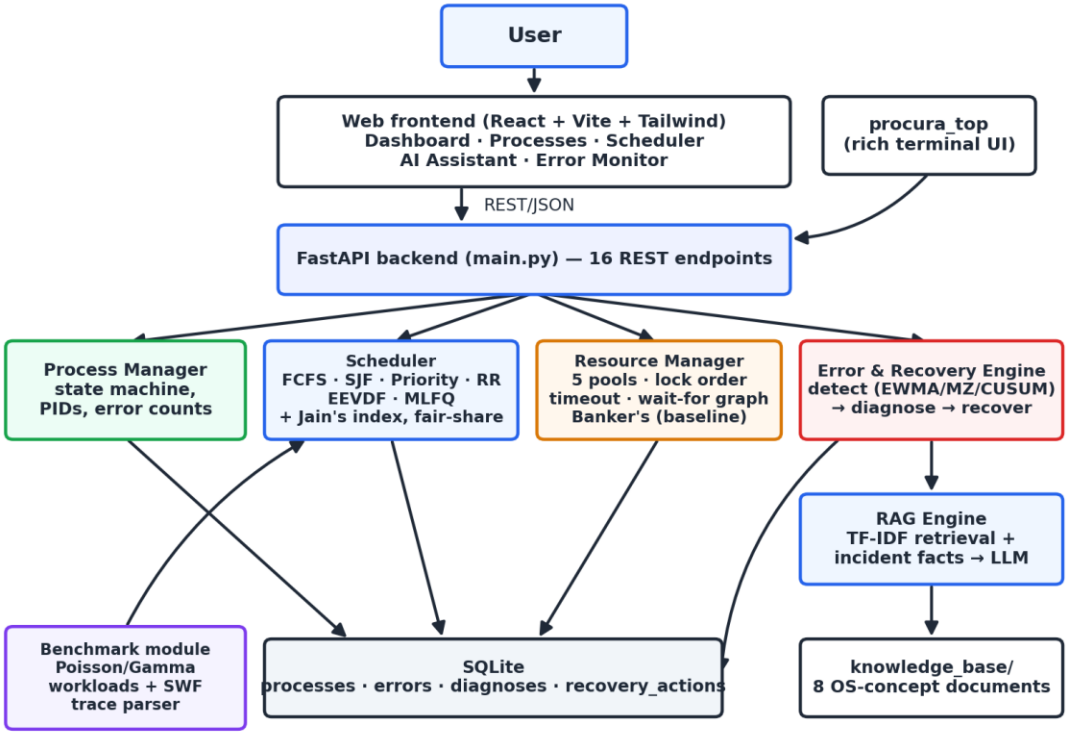


Figure 1. Procura architecture.

Table 1. Subsystems and their responsibilities.

Component	Module	Responsibility
Process Manager	`os_sim/process_manager.py`, `states.py`	PID assignment, the lifecycle state machine, error counts
Scheduler	`os_sim/scheduler.py`	Six policies, effective priority, fair-share, uniform metrics
Resource Manager	`os_sim/resource_manager.py`	Five resource pools, wait-for graph, lock order, timeouts, cycle detection, Banker's baseline
Error & Recovery Engine	`recovery/detector.py`, `diagnosis.py`, `recovery.py`	Detection, grounded diagnosis, deterministic recovery
RAG Engine	`rag/ingestion.py`, `retrieval.py`, `assistant.py`	TF-IDF index over the knowledge base; prompt assembly; LLM call or offline fallback
Benchmark	`benchmark/`	Workload generation, SWF parsing, metric computation
Interfaces	`main.py`, `frontend/`, `cli/procura_top.py`	REST API, five-screen web UI, terminal dashboard

Four design principles run through the system. It is **local-first**: it needs no account and no network unless a language-model key is configured. It is **offline-capable**: without a key the assistant returns a clearly labelled extractive answer built from retrieved passages. **Remediation is deterministic**: the language model explains failures but never chooses or executes actions. And **scope is deliberately narrow**: there is no authentication, container orchestration or job queue, which keeps the system runnable by one developer in minutes but makes it unsuitable for exposure on a shared network as it stands.

The implementation is about 4,900 lines of Python and JavaScript, of which 878 are tests. The backend uses Python 3, FastAPI and the standard-library `sqlite3` module; the frontend uses React 18, Vite and Tailwind CSS; the terminal dashboard uses `rich`. The statistical detectors and Jain's index use only the Python standard library.

4. Process model

Each task is a process with a PID, name, description, priority (High, Medium, Low or a 0–100 score), optional deadline, burst time (estimated minutes of work), arrival time, memory and CPU requirements, state, and error count. The state machine in Figure 2 is enforced on the server: any transition not listed in Table 2 raises an error that the API returns as HTTP 409, and the web UI offers only the actions valid for each state.

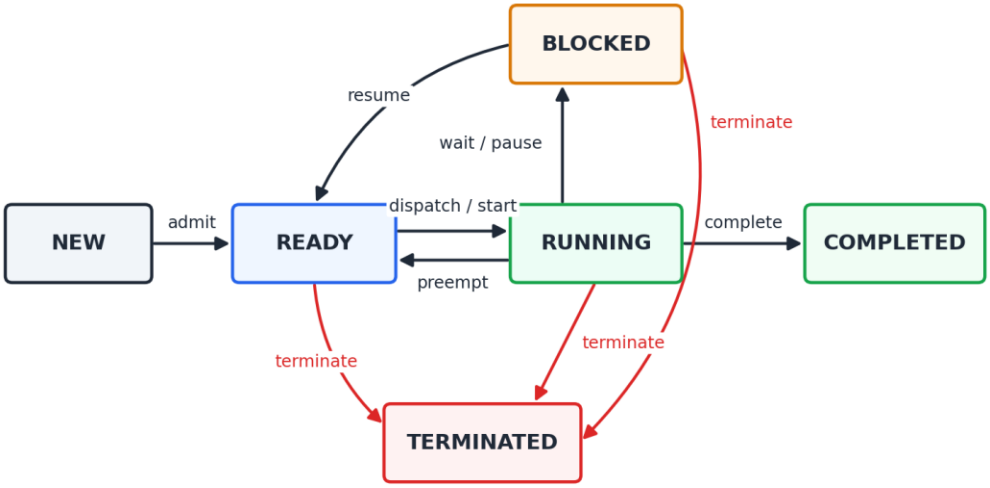


Figure 2. Process lifecycle state machine.

Table 2. Valid state transitions.

From	Allowed targets	Trigger
NEW	READY	Admission on creation
READY	RUNNING, TERMINATED	Dispatch or start; user or recovery termination
RUNNING	READY, BLOCKED, COMPLETED, TERMINATED	Preemption; wait or pause; completion; termination
BLOCKED	READY, TERMINATED	Resume; termination
COMPLETED	—	Terminal state
TERMINATED	—	Terminal state

State is persisted in four SQLite tables: `processes`; `errors` (one row per detected failure, with its raw details); `diagnoses` (the explanation generated for each error); and `recovery_actions` (the action type and target PID for each error). `GET /errors` joins the last three into the audit log behind the Error Monitor.

5. Scheduling

All six policies take the same list of processes, simulate a single CPU on a timeline measured in minutes, and return the same result: a dispatch order, a Gantt chart (a list of slices with PID, start and end), and the metrics below. Arrival times are taken relative to the earliest arrival in the set.

5.1 Metrics

For process *i* with arrival time A_i, burst B_i, completion time C_i (end of its last slice) and first dispatch F_i:

$$\text{Turnaround}_i = C_i - A_i \qquad \text{Waiting}_i = \text{Turnaround}_i - B_i \qquad \text{Response}_i = F_i - A_i$$

$$\text{Utilisation} = (\Sigma \text{ slice durations}) \,/\, \text{makespan} \times 100\%, \quad \text{makespan} = \max \text{ end time}$$

$$J(x) = (\Sigma\, x_i)^2 \,/\, (n \cdot \Sigma\, x_i^2), \quad x_i = \text{CPU time allocated to process } i$$

Averages are taken over all *n* processes. Jain's index is computed for every policy over each process's total allocated CPU time; Section 11 discusses why this makes it identical across work-conserving policies.

5.2 FCFS, SJF and Priority

All three are non-preemptive. FCFS dispatches in arrival order. SJF and Priority choose, at each decision point, the best process **among those that have already arrived** — the shortest burst for SJF, the highest effective priority for Priority — and the CPU idles only if nothing has arrived. Ties are broken by arrival time and then PID. An earlier version sorted the whole list up front, which left the CPU idle waiting for late arrivals (Section 9).

5.3 Effective priority

The Priority policy ranks processes by a five-term score:

$$E = U + D + W + R + S$$

- **User priority U:** High 90, Medium 50, Low 20; a numeric priority is clamped to 0–100.
- **Deadline urgency D:** 0 with no deadline or more than 48 hours remaining; 100 once the deadline has passed; otherwise `100 × (1 − h/48)` for *h* hours remaining.
- **Waiting-time aging W:** `min(100, 2 × minutes in READY)`, which prevents indefinite starvation.
- **Resource requirement R:** `min(60, mem/2048 × 60) + min(40, cpu/100 × 40)`; heavy tasks rank slightly higher because they block more of the system while they wait.
- **Fair-share S:** zero or negative, described next.

5.4 Slurm-style fair-share

Each process may carry the CPU time it has already consumed and when that was measured. Usage decays exponentially with a 60-minute half-life, by analogy with Slurm's `PriorityDecayHalfLife` [8]:

$$u_i = \text{consumed}_i \times 0.5\text{^}(\text{age}_i \,/\, 60)$$

With total decayed usage *T* over *n* processes, the fair share is *T/n*. A process using more than its share receives the penalty

$$S_i = -\min(100, ((u_i - T/n) / T) \times 100 \times n)$$

and a process with no recorded usage, the common case for new tasks, receives 0.

5.5 Round Robin

Processes enter a FIFO ready queue in arrival order; each dispatch runs for at most a quantum *q* (4 minutes by default), and processes arriving during a slice are queued before the preempted process is re-queued. The wait between consecutive turns is bounded by (n − 1)·q.

5.6 EEVDF

Priority maps to a kernel-style nice value (High −5, Medium 0, Low +5; a 0–100 score maps linearly to +10…−10) and then to a weight $w_i = 1024 / 1.25^{nice_i}$. Each process carries a virtual runtime v_i, initially zero. At every decision point the scheduler computes the weight-averaged virtual runtime of the active set; a process is eligible when its lag, the average minus v_i, is non-negative. Among eligible processes, a min-heap returns the one with the earliest virtual deadline:

$$\text{deadline}_i = v_i + s \cdot 1024 / w_i$$

The chosen process runs for the base slice *s* (the UI's quantum, floored at 3 minutes) or its remaining burst, whichever is shorter, after which

$$v_i \leftarrow v_i + \text{run} \cdot 1024 / w_i$$

Every slice therefore has the same length in real time; weighting acts through virtual time, which advances more slowly for heavier processes, so they become eligible, and are dispatched, more often. Low-weight processes cannot starve because their lag grows until they become eligible. This is a faithful simulation of EEVDF's eligibility and deadline rules, not a port of the kernel; per-task requested slices are not modelled.

5.7 Multi-Level Feedback Queue

MLFQ uses three queues with quanta of *q*, 2*q* and 4*q* (4, 8 and 16 minutes by default) and implements OSTEP's five rules [4]. A process's allotment at a level equals that level's quantum and is accounted **cumulatively**: being preempted does not reset it, which closes the exploit in which a job yields just before its quantum expires to stay at high priority. Exhausting the allotment demotes the process one level, and every 50 minutes of simulated time all processes return to the top queue.

6. Resource management and deadlock handling

Table 3. Simulated resource pools.

Resource	Capacity	Unit
CPU	100	percent
Memory	2048	MB
File	64	handles
Database	20	connections
Network	1000	Mbps

An allocation request succeeds if the pool has enough free capacity. If it fails because another process holds the resource, the manager records a wait-for edge from the requester to the holder. Releasing resources removes the corresponding edges. Procura then provides four mechanisms that together reproduce the practical spectrum of Section 2.5:

- **Lock-order checking (default on).** Resources have a canonical order: CPU, memory, file, database, network. A process that requests a resource while holding one later in that order is logged as a lock-order violation, as Linux's lockdep does. Violations are logged, not blocked.
- **Timeout-based abort.** Any wait that exceeds 30 seconds is aborted and logged, in the manner of MySQL's `innodb_lock_wait_timeout`.
- **Wait-for-graph cycle detection.** A three-colour depth-first search returns the PIDs on any cycle. Recovery terminates the lowest-priority process in the cycle, releases its resources, and returns the other blocked members to READY.
- **Banker's-style avoidance (default off).** Retained as the textbook baseline: a request is granted only if, afterwards, at least one other process's declared need could still be met. It can be enabled through `POST /resources/avoidance-mode`.

7. Detection, diagnosis and recovery

`POST /detect` runs every detector once, records all findings, and then, for each finding in turn, generates a diagnosis, executes a recovery script, and logs all three. Because findings are recorded before any recovery runs, a later finding can be stale by the time it is handled; the recovery rules account for this.

7.1 Detection

Memory-allocation failure. For each process that is not COMPLETED or TERMINATED and does not already hold its full requirement, the detector reports a failure when the remaining requirement exceeds available memory.

Deadlock. A cycle in the wait-for graph, found as described in Section 6.

Burst-time anomaly. For each RUNNING process, the detector computes the ratio of elapsed running time to declared burst time and compares it with that process's last 30 ratios using one of three NIST methods. EWMA is the default.

$$\mathrm{EWMA}_t = \lambda \cdot Y_t + (1-\lambda)\cdot \mathrm{EWMA}_{t-1}, \quad \text{limits: } \mathrm{EWMA}_0 \pm k\cdot\sigma\cdot\sqrt{(\lambda\,/\,(2-\lambda))}, \quad \lambda = 0.25,\ k = 3$$

$$M_i = 0.6745\,(x_i - \text{median})\,/\,\mathrm{MAD}, \quad \text{flag if } |M_i| > 3.5$$

$$S_hi = \max(0,\ S_hi + x - \mu - k), \quad S_lo = \max(0,\ S_lo - x + \mu - k), \quad \text{flag if either exceeds } h$$

For CUSUM, *k* corresponds to a one-standard-deviation shift and *h* to five standard deviations. The original rule — running time above 1.5 times the burst while CPU utilisation is at least 80% — is kept as a `fixed_threshold` mode for comparison.

7.2 Grounded diagnosis

The knowledge base consists of eight plain-text documents (Table 4). Ingestion splits each on paragraph boundaries into chunks of at most about 600 characters — 63 chunks in total — and builds a TF-IDF index over a vocabulary of 1,422 terms, with smoothed inverse document frequency

$$\mathrm{idf}(t) = \ln((1 + N) / (1 + \mathrm{df}(t))) + 1$$

where *N* is the number of chunks. A query is vectorised the same way and the three chunks with the highest cosine similarity are retrieved, optionally restricted to the documents relevant to the failure type (for example, Memory_Management and Error_Recovery for a memory failure).

Table 4. Knowledge-base documents.

Document	Chunks	Scope
CPU_Scheduling	7	FCFS, SJF, Priority, Round Robin, metrics
Advanced_Scheduling	16	CFS, EEVDF, MLFQ, aging, Jain's index, Slurm fair-share
Process_Management	5	Process states, PCBs, context switches
Memory_Management	5	Allocation, fragmentation, allocation failure
Deadlocks	9	Coffman conditions; prevention, avoidance, detection; production practice
File_System	5	File-system fundamentals
Error_Recovery	9	Detect-diagnose-recover, runbooks, Procura's preemption rules, grounding
Anomaly_Detection	7	Fixed thresholds versus EWMA, modified Z-score and CUSUM

The prompt has four parts: an instruction, the process record, an **incident-facts** section, and the retrieved passages. The incident facts list the error details and executed recovery actions recorded for the process, and the current memory state, explicitly labelled as current rather than as at the time of failure. The instruction tells the model to treat these facts as ground truth and to use the passages to explain concepts rather than to propose causes the facts do not show. The prompt is sent to the first configured provider — Groq (`openai/gpt-oss-20b`), Anthropic or OpenAI. Without a key, or if the call fails, the assistant returns the retrieved passages and the incident facts verbatim, labelled as an offline extractive answer.

7.3 Recovery

Table 5. Failure types and recovery scripts.

Failure	Script	Effect
memory_failure	retry / suspend_and_retry / memory_failure_no_victim	Decision flow of Figure 3
deadlock	terminate_and_resume	Terminate the lowest-priority process in the cycle, release its resources, return the others to READY
burst_anomaly, timeout	stop_and_retry	Move the process back to READY and increment its error count
crash	restart	Move the process back to READY and increment its error count

Memory-failure recovery follows the decision flow in Figure 3, modelled on Kubernetes preemption [11]. It first retries, because memory freed by an earlier recovery in the same pass may already suffice. Otherwise it considers the lowest-priority memory holder *V*, and suspends it only if *V* has **strictly lower** effective priority than the failing process *F* and releasing *V*'s memory would actually let *F*'s allocation fit. If either condition fails, *F* is left pending and nothing is evicted. Every memory failure increments *F*'s error count.

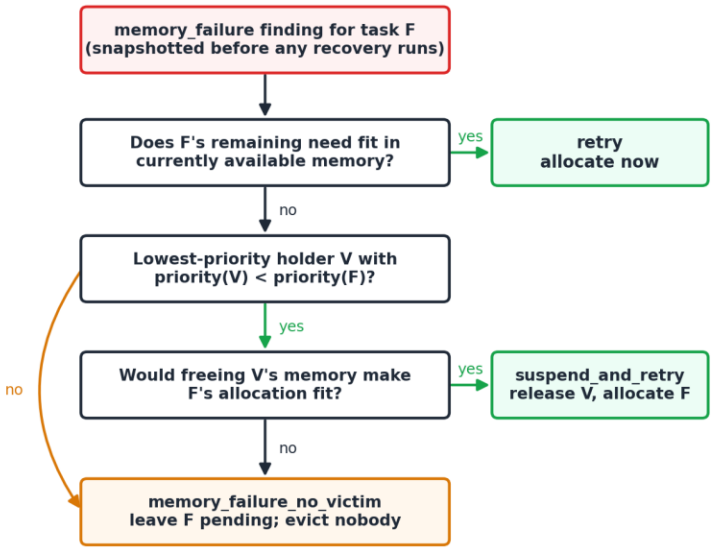


Figure 3. Recovery decision flow for a memory-allocation failure.

8. Interfaces and configuration

Table 6. REST API.

Method	Endpoint	Purpose
GET	/health	Liveness check
GET, POST	/processes	List processes; create a process
GET, PATCH, DELETE	/processes/{pid}	Read; transition (start, pause, resume, complete, terminate); delete
GET, POST	/schedule	Run a policy; returns order, Gantt data, metrics and Jain's index
GET	/benchmark	Run all six policies on a synthetic workload
GET	/resources	Utilisation per pool and the wait-for graph
POST	/resources/allocate, /resources/release	Allocate or release a resource for a process
POST	/resources/avoidance-mode	Enable or disable Banker's-style avoidance
POST	/detect	Run detection, diagnosis and recovery once
POST	/ask	Ask the assistant, optionally grounded on a PID
GET	/errors	Full audit log of errors, diagnoses and recovery actions

The web interface has five screens: a Dashboard (process table, resource bars and AI activity feed, refreshed every few seconds), Processes (creation form and state-driven actions), Scheduler (policy selector, Gantt chart with one lane per process, metrics), AI Assistant, and Error Monitor. `procura_top` renders the same process table, resource bars and activity feed in a terminal. Figure 4 shows the scheduler screen; the companion demo document contains screenshots of every screen.

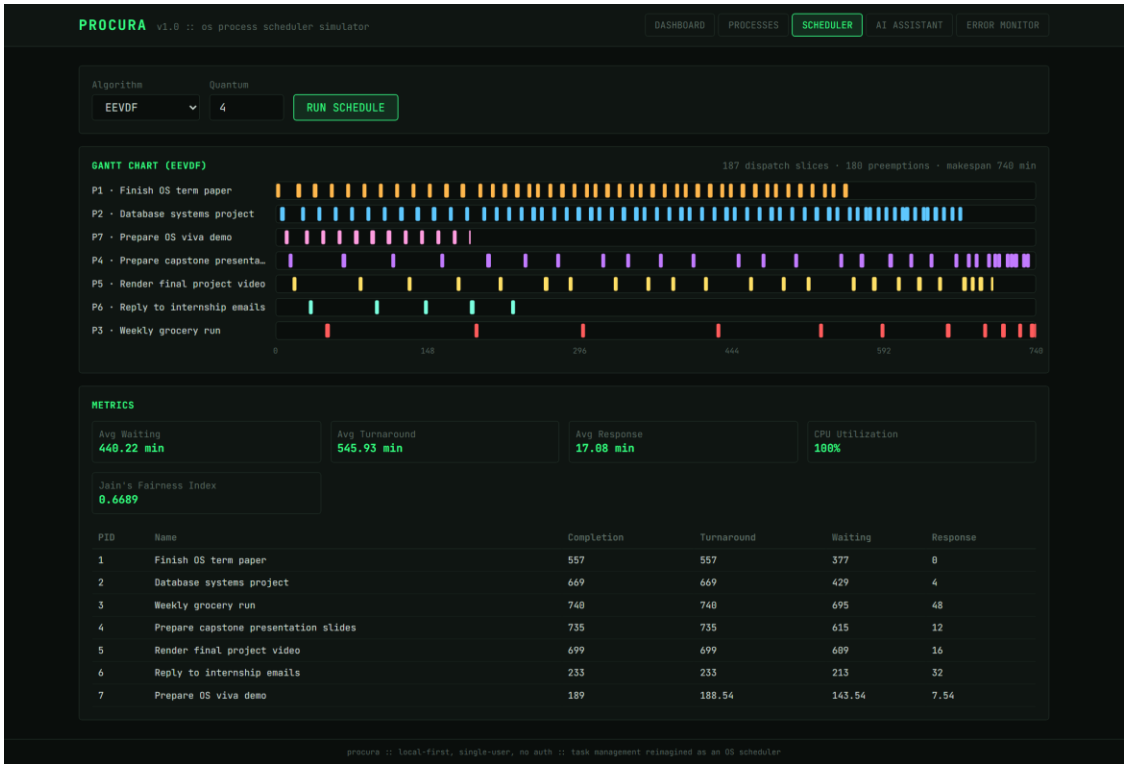


Figure 4. The Scheduler screen running EEVDF on seven tasks: 187 dispatch slices, with high-priority tasks (top lanes) dispatched far more often than the low-priority task in the bottom lane.

Table 7. Configuration.

Variable	Purpose	Default
GROQ_API_KEY, ANTHROPIC_API_KEY, OPENAI_API_KEY	Language-model provider, checked in this order	Unset (offline mode)
VITE_API_URL	Backend URL used by the web frontend	http://localhost:8000
PROCURA_API_URL	Backend URL used by procura_top	http://localhost:8000

9. Verification

Table 8. Automated test suite (83 tests, all passing).

Module	Tests	Coverage
test_scheduler.py	18	All six policies; hand-worked FCFS, SJF and Round Robin examples; arrival-aware SJF and Priority; effective priority; fair-share; Jain's index
test_recovery.py	19	Detectors (positive and negative cases for each method), recovery scripts, preemption rules, error counting, finished-process filtering
test_resource_manager.py	13	Allocation, over-capacity rejection, wait-for edges, cycles, lock order, timeouts, avoidance mode
test_api.py	11	Endpoint smoke tests, invalid-transition 409, grounding of answers in incident facts
test_benchmark.py	8	SWF parsing, seeded generation, hand-computed makespan, utilisation and starvation
test_process_manager.py	7	Lifecycle transitions and error counting
test_retrieval.py	7	Each implemented concept retrieves the correct knowledge-base document

Tests are hermetic: the test configuration blanks the provider keys before the application loads its environment file, so no test makes a network call. The SJF and Round Robin tests assert exact completion and waiting times from hand-worked examples. For four processes arriving at minutes 0, 1, 2 and 3 with bursts of 5, 3, 8 and 6, SJF must run P1 first — it is the only arrival at time 0 — and then P2, P4 and P3, giving an average waiting time of 5.25 minutes.

9.1 Defects found by exercising the running system

Unit tests passed throughout, yet capturing screenshots of every screen and reading the output closely revealed twelve defects (Table 9). Two lessons stand out. First, a test can encode a defect: the original SJF test asserted the idle-CPU behaviour (average wait 5.75) and so protected the bug; that defect also inverted the published benchmark conclusion, making SJF look worse than FCFS when it is in fact the best policy on average waiting time. Second, the assistant's errors were failures of grounding, not of the model: given the recorded facts and relevant passages, the same model gave correct answers.

Table 9. Defects found, root causes and fixes.

#	Symptom	Root cause	Fix and guard
1	SJF and Priority makespan 314.6 and 304.0 vs 228.7 for FCFS; utilisation below 100%	Whole list pre-sorted; CPU idled waiting for late arrivals	Choose among arrived processes; corrected test; regression test
2	Gantt chart had ~190 near-empty rows for preemptive policies	One chart row per slice instead of per process	One lane per process with slices positioned on a shared time axis
3	Assistant blamed memory fragmentation; log showed 1600 MB requested, 248 MB free	Prompt contained the process record but no incident data	Incident facts injected as an authoritative prompt section; API test
4	Assistant expanded EEVDF as 'Effective-Priority Earliest-Deadline-First'	Knowledge base predated EEVDF, MLFQ, fair-share and the SPC methods	Two new documents, two extended; seven retrieval tests
5	Two processes evicted each other repeatedly	Victim chosen without comparing priorities	Evict only strictly lower-priority holders; test
6	A process was suspended without freeing enough memory to help	No check that eviction makes the allocation fit	Fit check before eviction; test
7	Failures handled after memory had already been freed	Findings recorded before recovery and acted on unchanged	Retry first; test
8	Processes flagged although they already held their memory	Detector compared the full requirement with free memory	Compare only the unmet remainder; test
9	Finished processes could be flagged and 'recovered'	Detector ran over every process	Skip COMPLETED and TERMINATED; test
10	Error count stayed 0 after memory failures	Only timeout and crash recovery incremented it	Increment on every memory failure; test
11	Test suite made live LLM calls	Keys removed from the environment were re-added by load_dotenv	Blank the keys instead of removing them
12	Application unusable when port 8000 was taken	Backend URL hard-coded in frontend and CLI	VITE_API_URL and PROCURA_API_URL settings

10. Evaluation

10.1 Method

The benchmark module generates synthetic workloads with Poisson arrivals (exponential inter-arrival times) and Gamma-distributed burst times, a simplification of the Lublin–Feitelson model [37], and can instead replay a trace in the Standard Workload Format [36]. Every policy runs on the identical job list. In addition to the metrics of Section 5.1 it reports maximum waiting time and, because starvation has no standard definition [43], the number of jobs that waited more than 30 minutes. Generation is seeded, so `python -m benchmark.run_benchmark --n 30 --arrival-rate 0.3 --seed 42` reproduces Table 10 exactly. We also replay a hand-written 15-job SWF sample and report the live seven-task demonstration captured for the companion document.

10.2 Results

Table 10. Synthetic workload: 30 jobs, Poisson arrivals at 0.3 per minute, seed 42 (minutes).

Policy	Makespan	Avg turnaround	Avg waiting	Max waiting	Waited > 30 min	Jain's index
FCFS	228.67	69.19	61.56	133.50	23	0.6711
SJF	228.67	41.65	34.03	163.10	8	0.6711
Priority	228.67	68.74	61.12	214.83	15	0.6711
Round Robin	228.67	82.71	75.09	161.05	27	0.6711
EEVDF	228.67	80.18	72.56	204.51	18	0.6711
MLFQ	228.67	88.00	80.38	175.10	27	0.6711

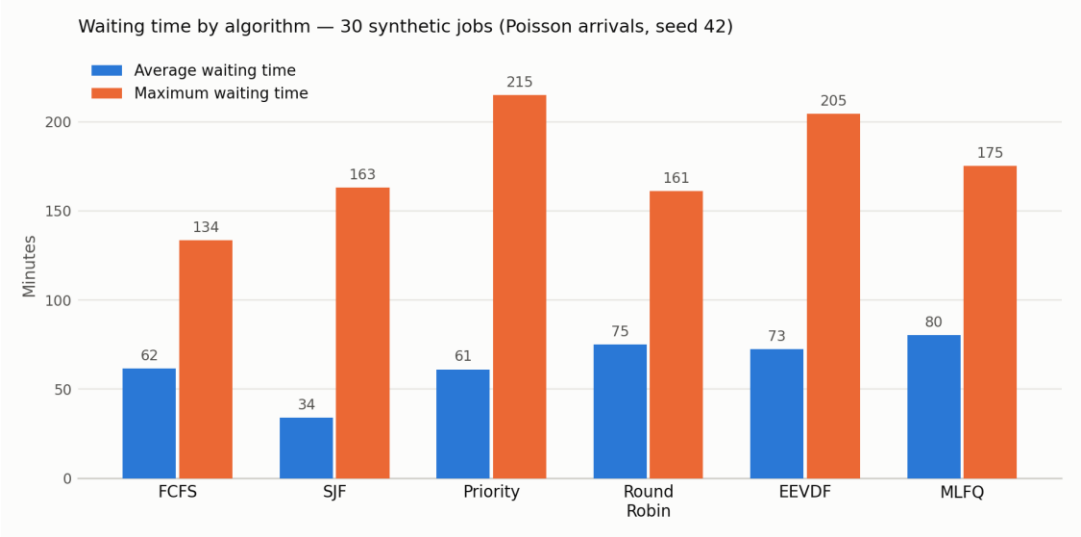


Figure 5. Average and maximum waiting time per policy on the synthetic workload (data in Table 10).

Table 11. SWF trace replay: 15 jobs (minutes).

Policy	Makespan	Avg turnaround	Avg waiting	Max waiting	Waited > 30 min	Jain's index
FCFS	2305.0	1115.33	961.67	1905.0	14	0.465
SJF	2305.0	493.33	339.67	1555.0	11	0.465
Priority	2305.0	1553.67	1400.00	2170.0	14	0.465
Round Robin	2305.0	878.60	724.93	1555.0	15	0.465
EEVDF	2305.0	874.00	720.33	1555.0	12	0.465
MLFQ	2305.0	884.47	730.80	1555.0	15	0.465

Table 12. Live demonstration: seven tasks, quantum 4 minutes (minutes; makespan 740 for every policy).

Policy	Slices	Preemptions	Avg waiting	Avg turnaround	Avg response
FCFS	7	0	431.37	537.09	431.37
SJF	7	0	173.51	279.23	173.51
Priority	7	0	436.37	542.09	436.37
Round Robin	187	180	348.94	454.66	11.94
EEVDF	187	180	440.23	545.94	17.09
MLFQ	110	103	368.37	474.09	11.94

10.3 Discussion

All six policies are work-conserving. None leaves the CPU idle while a job is waiting, so makespan and utilisation are identical and are determined by the workload alone. The policies differ only in who waits and for how long.

SJF minimises average waiting time but not the worst case (Figure 5). SJF has the lowest average waiting time on both workloads (34.03 against 61.56 for FCFS on the synthetic run; 339.67 against 961.67 on the trace) and the fewest jobs over the 30-minute threshold, as theory predicts. Its maximum wait (163.1) is nevertheless higher than FCFS's (133.5), because long jobs are repeatedly overtaken — the starvation risk of Section 2.1.

Preemptive policies trade waiting time for responsiveness. On the synthetic workload Round Robin, EEVDF and MLFQ all have higher average waiting time than SJF. Their advantage is response time: in the live demonstration, Round Robin and MLFQ start every task within about 12 minutes on average, against 173.5 for SJF and 431.4 for FCFS. EEVDF's average response (17.1) is slightly higher because it deliberately favours heavy-weight tasks: the High-priority term paper starts at minute 0 while the Low-priority errand waits about 48 minutes. Among the preemptive policies EEVDF had the lowest average wait and fewest starved jobs on the synthetic run.

Effective priority behaves as specified but encodes value judgements. In the live run, a Medium-priority task ranked above a High-priority one because both deadlines had passed, giving each the maximum urgency, and the Medium task's larger resource requirement added more (225.9 against 218.0), while a High-priority task without a deadline ranked sixth. The behaviour is correct for the formula; whether these weights match what a user wants is a design question rather than a property of the scheduler.

10.4 Self-healing scenario

To exercise recovery, the low-priority errand (P3) was allocated 1200 MB and the High-priority term paper (P1) 600 MB, leaving 248 MB of 2048. One detection pass reported four memory failures, all recorded with 248 MB available because findings are recorded before any recovery runs. Table 13 shows how each was handled; the final state, 1880 MB allocated (600 + 768 + 256 + 256), matches what the dashboard reported.

Table 13. Recovery outcomes in the self-healing scenario.

Process	Needs	Outcome	Reason
P2 (High)	768 MB	suspend_and_retry, victim P3	P3 has strictly lower priority, and its 1200 MB makes P2 fit
P4 (Medium)	256 MB	retry	680 MB were free after P2's recovery
P5 (Medium)	1600 MB	memory_failure_no_victim	No lower-priority holder could free enough memory; nothing evicted
P7 (High)	256 MB	retry	424 MB were free after P4's allocation

Before the preemption rules of Section 7.3, the same scenario produced a cascade in which tasks evicted each other and one was suspended without freeing enough memory to help.

10.5 Grounding of the assistant

We did not run a quantitative evaluation of answer quality; we report two observed cases. Asked why P5 failed, the assistant originally attributed the failure to memory fragmentation; after incident facts were added to the prompt, it stated the recorded numbers and correctly explained that no process was evicted because none had strictly lower priority and would free enough memory. Asked how EEVDF differs from Round Robin, it originally invented an expansion of the acronym; after the knowledge base was extended it correctly described lag, eligibility, virtual deadlines and weighted sharing, though it still overstated two points (that Round Robin can leave a job unable to finish, and that Linux moved to EEVDF from Round Robin rather than from CFS). In both cases the deciding factor was what the retrieval and prompt supplied, which matches the finding that retrieval over structured state outperforms flat text retrieval [21].

11. Limitations and threats to validity

- **Simulation, not an operating system.** Time is an abstract minute scale on a single simulated CPU; nothing schedules real threads, and the EEVDF and MLFQ implementations follow the published rules without reproducing kernel internals.
- **Jain's index over total allocation.** Every work-conserving policy eventually gives each process exactly its burst time, so the index is identical across policies for a given workload. It measures how much each process receives, not when; waiting and response time carry the real differences.
- **Workloads.** The synthetic generator uses a Gamma distribution rather than the calibrated Lublin–Feitelson model, the SWF replay used a 15-job hand-written sample rather than an archive trace, and the live demonstration has seven tasks. Results show the expected qualitative behaviour but are not a statistical comparison.
- **The assistant was evaluated qualitatively.** Model output varies between runs and can still overstate; answers should be read alongside the audit log. A quantitative evaluation would need a labelled question set.

- **Burst-time anomaly detection was not exercised end to end in the demonstration,** because the detector only examines RUNNING processes and the scenario used READY ones; it is covered by unit tests.
- **Suspension does not change state** unless the victim was RUNNING; the victim's memory is released but it remains READY.
- **No authentication.** The system is designed for a single local user and must not be exposed on a shared network without adding access control.
- **Interface details.** The assistant's markdown is displayed unrendered, and the terminal dashboard clips its memory label.

12. Future work

- Report fairness over waiting time and allocation rate, not only total allocation; sweep quanta, boost intervals and slice lengths and report sensitivity.
- Replay full Parallel Workloads Archive traces and implement the calibrated Lublin–Feitelson generator.
- Build a labelled question set to measure the assistant's factual accuracy, and compare TF-IDF retrieval with embedding-based retrieval while keeping the offline path.
- Add multi-user operation with authentication so that fair-share applies across real users, and let tasks request their own EEVDF slice.
- Enforce lock ordering rather than only logging violations, and add a distributed detection algorithm such as Chandy–Misra–Haas as an option.

13. Conclusion

Procura shows that the scheduling, fairness, resource-management and self-healing mechanisms of production systems can be applied faithfully to personal task management, and that doing so makes their trade-offs visible: SJF minimises average waiting at the cost of the longest tasks, preemptive policies buy responsiveness with waiting time, EEVDF turns priority into weighted sharing without starvation, and principled preemption rules turn a cascading recovery engine into one whose every action can be justified. The verification account is the other contribution. Twelve defects survived a passing unit-test suite and were found only by running the system and reading its output, one of them inverting a published result. The assistant's errors, in turn, came from missing context rather than from the model, and disappeared when the facts and the knowledge were supplied. Both point to the same practice: check results against the running system and against theory, and report what is found.

References

[1] Linux kernel documentation, "EEVDF Scheduler." https://docs.kernel.org/scheduler/sched-eevdf.html

[2] LWN.net, article 925371 on the EEVDF scheduler. https://lwn.net/Articles/925371

[3] UC Berkeley CS162, Lecture 13, Fall 2023. https://inst.eecs.berkeley.edu/~cs162/fa23/static/lectures/13.pdf

[4] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, Operating Systems: Three Easy Pieces, ch. 8, "Scheduling: The Multi-Level Feedback Queue." https://fog.misty.com/perry/os/Chapters/08-cpu-sched-mlfq/all-notes.html

[5] UC Berkeley CS162, Lectures 10–11, Fall 2023. https://inst.eecs.berkeley.edu/~cs162/fa23/static/lectures/10.pdf

[6] R. Jain, D. Chiu and W. Hawe, "A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems," DEC Technical Report TR-301, 1984. https://classes.engineering.wustl.edu/~jain/papers/fairness.htm

[7] R. Jain et al., fairness index (republished), arXiv cs/9809099. https://arxiv.org/pdf/cs/9809099

[8] University of Bayreuth HPC, "Scheduling and Priority" (Slurm multifactor priority). https://www.hpc.uni-bayreuth.de/advanced/schedandprio/

[9] SchedMD, "Slurm Scheduling Configuration Guide." https://slurm.schedmd.com/sched_config.html

[10] Kubernetes v1.33 documentation, "Scheduling Framework." https://v1-33.docs.kubernetes.io/docs/concepts/scheduling-eviction/scheduling-framework/

[11] Kubernetes v1.33 documentation, "Scheduler Configuration." https://v1-33.docs.kubernetes.io/docs/reference/scheduling/config

[12] Celery documentation, "Routing Tasks." https://docs.celeryq.dev/en/main/userguide/routing.html

[13] Google, Site Reliability Engineering, "Eliminating Toil." https://sre.google/sre-book/eliminating-toil/

[14] Kubernetes documentation, "Controllers." https://kubernetes.io/docs/concepts/architecture/controller/

[15] Datadog, "Watchdog." https://docs.datadoghq.com/watchdog/

[16] Dynatrace, "Davis AI root cause analysis." https://docs.dynatrace.com/docs/platform/davis-ai/root-cause-analysis

[17] New Relic, "Announcing New Relic AI general availability." https://newrelic.com/blog/news/announcing-new-relic-ai-general-availability

[18] Microsoft Research, RCACopilot: LLM-based root cause analysis for cloud incidents, EuroSys 2024, arXiv:2305.15778. https://arxiv.org/abs/2305.15778

[19] Microsoft Research, in-context learning with GPT-4 for root cause analysis over 100,000+ cloud incidents, arXiv:2401.13810. https://arxiv.org/pdf/2401.13810

[20] ReAct-style LLM agent for root cause analysis, arXiv:2403.04123. https://arxiv.org/pdf/2403.04123

[21] SynergyRCA: retrieval over graph databases with GPT-4o for Kubernetes root cause analysis, arXiv:2506.02490. https://arxiv.org/pdf/2506.02490

[22] PagerDuty, "Rundeck Runbook Automation." https://www.pagerduty.com/integrations/rundeck-runbook-automation/

[23] AWS, "Working with runbooks in Systems Manager Application Manager." https://docs.aws.amazon.com/systems-manager/latest/userguide/application-manager-working-runbooks.html

[24] AWS, "Remediation impact types." https://docs.aws.eu/systems-manager/latest/userguide/remediation-impact-type.html

[25] NIST/SEMATECH e-Handbook of Statistical Methods, §6.3.2.4, "EWMA Control Charts." https://www.itl.nist.gov/div898/handbook/pmc/section3/pmc324.htm

[26] NIST/SEMATECH e-Handbook of Statistical Methods, §1.3.5.17, "Detection of Outliers." https://www.itl.nist.gov/div898/handbook/eda/section3/eda35h.htm

[27] NIST/SEMATECH e-Handbook of Statistical Methods, §6.3.2.3, "CUSUM Control Charts." https://www.itl.nist.gov/div898/handbook/pmc/section3/pmc323.htm

[28] SolarWinds, "4 Statistical Process Control Rules That Detect Anomalies in Systems." https://www.solarwinds.com/blog/4-statistical-process-control-rules-that-detect-anomalies-in-systems

[29] Prometheus documentation, "Query functions." https://prometheus.io/docs/prometheus/latest/querying/functions/

[30] On the practical limitations of Banker's Algorithm, arXiv:2511.07071.

[31] Google Cloud, "Life of Cloud Spanner Reads & Writes." https://docs.cloud.google.com/spanner/docs/whitepapers/life-of-reads-and-writes

[32] LWN.net, "Wait/wound mutexes." https://lwn.net/Articles/548909

[33] MySQL 8.0 Reference Manual, "Deadlock Detection." https://dev.mysql.com/doc/refman/8.0/en/innodb-deadlock-detection.html

[34] PostgreSQL documentation, "Lock Management" (deadlock_timeout). https://www.postgresql.org/docs/current/runtime-config-locks.html

[35] K. M. Chandy, J. Misra and L. M. Haas, "Distributed Deadlock Detection," ACM TOCS, 1983. https://authors.library.caltech.edu/records/gsez6-b8b74

[36] D. Feitelson, Parallel Workloads Archive, "The Standard Workload Format." https://www.cs.huji.ac.il/labs/parallel/workload/swf.html

[37] U. Lublin and D. Feitelson workload model, as used in IPDPS 2006. https://www.cecs.uci.edu/~papers/ipdps06/pdfs/1568974798-IPDPS-paper-1.pdf

[38] Batsim documentation, "Output: schedule." https://batsim.readthedocs.io/en/main/output-schedule.html

[39] Scheduling metrics discussion, arXiv:2111.10200.

[40] A. Verma et al., "Large-scale cluster management at Google with Borg," EuroSys 2015. https://research.google/pubs/pub43438

[41] Google, cluster-data traces. https://github.com/google/cluster-data

[42] CloudSim toolkit for modelling and simulating cloud computing environments, arXiv:0903.2525. https://arxiv.org/pdf/0903.2525

[43] Starvation thresholds in GPU cluster scheduling, arXiv:2512.10980.

[44] Scaled Agile Framework, "WSJF." https://v5.scaledagileframework.com/?p=3408

Appendix A. Reproduction

```
# backend
cd backend
pip install -r requirements.txt
python seed_data.py                     # six demo tasks
uvicorn main:app --port 8000            # or another port

# frontend (second terminal)
cd frontend && npm install
VITE_API_URL=http://localhost:8000 npm run dev

# terminal dashboard, benchmark, tests
PROCURA_API_URL=http://localhost:8000 python -m cli.procura_top
python -m benchmark.run_benchmark --n 30 --arrival-rate 0.3 --seed 42
python -m benchmark.run_benchmark --swf benchmark/sample_traces/sample.swf
python -m pytest tests/ -v
```

Companion documents in `docs/`: the screenshot-based demo walkthrough, the case-study report, and the review presentation.